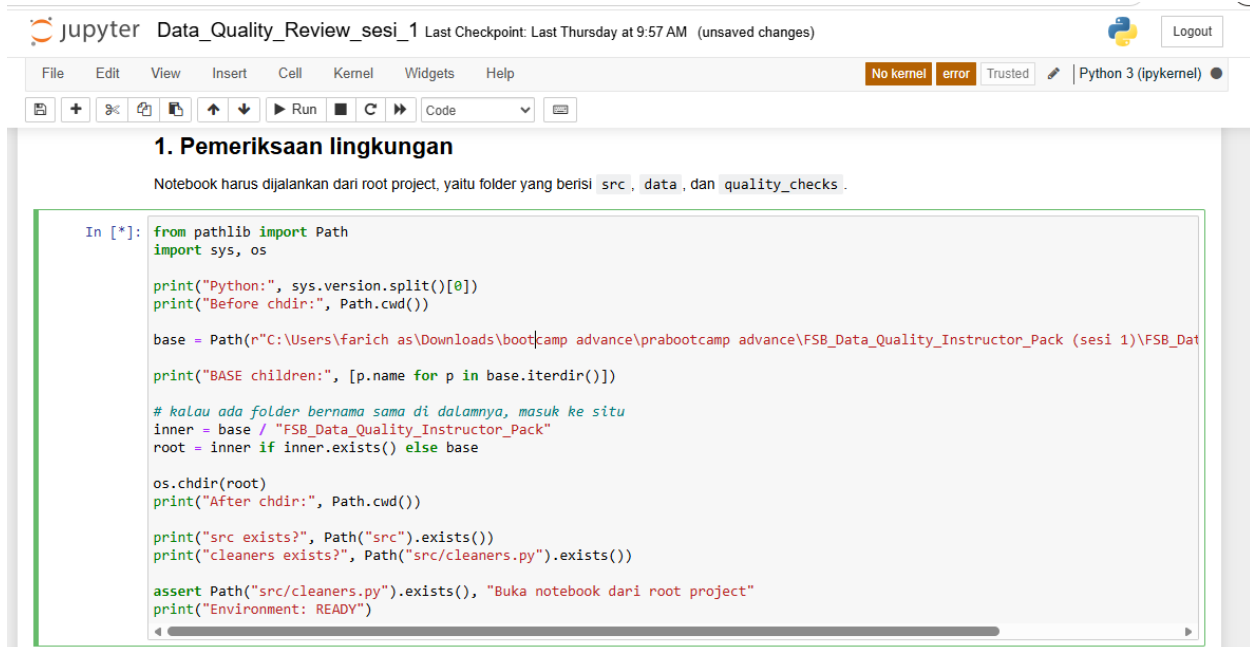


## 1. Data\_Quality\_Review

Script **Data\_Quality\_Review** berfungsi sebagai **pemeriksaan awal kualitas dan kondisi data/project** sebelum proses cleaning dan pipeline dijalankan. Inputnya adalah struktur project serta data mentah yang tersedia, terutama file-file pada folder `data/raw`. Prosesnya mencakup pemeriksaan keberadaan file, struktur data, jumlah record, dan kondisi awal data untuk mengetahui apakah data sudah tersedia dan layak digunakan sebagai bahan proses berikutnya. Outputnya berupa **informasi atau status hasil pemeriksaan kualitas data**, sehingga pengguna dapat mengetahui kondisi awal data dan masalah yang perlu diperhatikan sebelum masuk ke tahap cleaning.



```
In [*]: from pathlib import Path
import sys, os

print("Python:", sys.version.split()[0])
print("Before chdir:", Path.cwd())

base = Path(r"C:\Users\farich as\Downloads\bootcamp advance\prabootcamp advance\FSB_Data_Quality_Instructor_Pack (sesi 1)\FSB_Dat
print("BASE children:", [p.name for p in base.iterdir()])

# kalau ada folder bernama sama di dalamnya, masuk ke situ
inner = base / "FSB_Data_Quality_Instructor_Pack"
root = inner if inner.exists() else base

os.chdir(root)
print("After chdir:", Path.cwd())

print("src exists?", Path("src").exists())
print("cleaners exists?", Path("src/cleaners.py").exists())

assert Path("src/cleaners.py").exists(), "Buka notebook dari root project"
print("Environment: READY")
```

## 2. python preflight

Script **python preflight** berfungsi sebagai **pemeriksaan kesiapan environment dan struktur project** sebelum pembelajaran atau pipeline dijalankan. Inputnya adalah versi Python, struktur folder project, file `src/cleaners.py`, serta tiga file raw yaitu `users.json`, `products.json`, dan `transactions.json`. Prosesnya memeriksa apakah Python minimal versi 3.11, project berada pada struktur yang benar, file raw tersedia dan berisi data, serta module `src.cleaners` dapat di-import tanpa error. Outputnya berupa status **PASS/FAIL** untuk setiap pemeriksaan dan kesimpulan **READY TO LEARN** jika seluruh persyaratan terpenuhi atau **FIX SETUP BEFORE CLASS** jika terdapat masalah.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python preflight.py
PASS Python 3.11+ - 3.12.0
PASS Root project - D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack
PASS Raw users - 16 rows
PASS Raw products - 12 rows
PASS Raw transactions - 15 rows
PASS Import src.cleaners - D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack\src\cleaners.py

READY TO LEARN
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> |
```

3. `src/foundations.py`

Script `src/foundations.py` berfungsi menyediakan **fungsi-fungsi dasar/helper** yang menjadi fondasi untuk proses pengolahan dan validasi data, seperti membersihkan nama, mengambil nilai kota dengan fallback, mengklasifikasikan quantity, memberi nomor pada daftar nama, dan mengubah quantity menjadi integer positif. Inputnya berupa nilai sederhana seperti string, dictionary, integer, atau list yang diberikan kepada masing-masing fungsi. Prosesnya meliputi normalisasi teks, pengambilan nilai dictionary, klasifikasi tipe/nilai quantity, enumerasi data, dan konversi nilai menjadi angka yang valid. Outputnya berupa **nilai yang telah dinormalisasi, diklasifikasikan, diberi nomor, atau dikonversi**.

4. `src/cleaners.py`

Script `src/cleaners.py` merupakan **mesin utama data cleaning** yang berfungsi membersihkan dan memvalidasi tiga jenis data, yaitu users, products, dan transactions melalui fungsi `clean_users()`, `clean_products()`, dan `clean_transactions()`. Inputnya adalah record mentah dari `users.json`, `products.json`, dan `transactions.json`. Prosesnya mencakup normalisasi teks dan email, konversi angka, validasi ID, pengecekan duplikasi, pemeriksaan field wajib, pemberian nilai `NULL` pada field tertentu yang tidak valid tetapi record masih dapat dipertahankan, serta `DROP` terhadap record yang tidak layak; setiap keputusan juga dicatat dalam decision log. Output setiap fungsi adalah tiga bagian, yaitu **clean** untuk data yang berhasil dibersihkan, **rejected** untuk data yang ditolak, dan **decisions** untuk catatan keputusan cleaning.

5. `quality_checks/01_python_core.py`

Script `quality_checks/01_python_core.py` berfungsi sebagai **unit test untuk memastikan fungsi-fungsi dasar pada `src/foundations.py` bekerja sesuai kontrak yang telah ditentukan**. Inputnya berupa sejumlah test case yang berisi nilai input dan hasil yang diharapkan untuk fungsi seperti `clean_name()`, `safe_city()`, `classify_quantity()`, `numbered_names()`, dan `parse_quantity()`. Prosesnya menjalankan setiap fungsi, membandingkan hasil aktual dengan hasil yang diharapkan, kemudian menentukan apakah masing-masing kasus `PASS` atau `FAIL`. Outputnya berupa daftar hasil pengujian, jumlah test yang berhasil, serta exit code 0 jika seluruh test berhasil atau 1 jika terdapat kegagalan.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python quality_checks/01_python_core.py
PASS string assign => Rani
PASS dict existing key => Bandung
PASS dict fallback => Unknown
PASS quantity int => positive-int
PASS quantity string => numeric-string
PASS quantity invalid => invalid
PASS enumerate => [(1, 'Rani'), (2, 'Bima')]
PASS try except => [2, None, None]

8/8 core cases passed
```

6. python quality\_checks/02\_review\_input.py

Script `quality_checks/02_review_input.py` berfungsi melakukan **inspeksi terhadap data input/raw sebelum data tersebut diproses oleh cleaner**, sehingga pengguna dapat memahami bentuk dan struktur data yang akan digunakan. Inputnya adalah seluruh file JSON pada folder `data/raw`, seperti `users.json`, `products.json`, dan `transactions.json`. Prosesnya membaca setiap file, menentukan tipe container datanya, menghitung jumlah record, mengambil contoh record pertama, dan mengidentifikasi field yang tersedia. Outputnya berupa **informasi struktur data raw**, misalnya nama file, tipe data, jumlah record, contoh record pertama, dan daftar field, tanpa mengubah data sumber.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python quality_checks/02_review_inputs.py
products.json list 12
first {'product_id': 'P001', 'name': ' Laptop Stand ', 'price': '175000', 'category': 'Accessories'}
fields ['category', 'name', 'price', 'product_id']
transactions.json list 15
first {'tx_id': 'T001', 'user_id': 'U001', 'product_id': 'P001', 'quantity': 1, 'tx_date': '2026-08-01'}
fields ['product_id', 'quantity', 'tx_date', 'tx_id', 'user_id']
users.json list 16
first {'user_id': ' U001 ', 'name': ' Rani Putri ', 'email': 'RANI@MAIL.COM', 'city': 'Bandung'}
fields ['city', 'email', 'name', 'user_id']
```

7. python quality\_checks/03\_helper\_contracts.py

Script `quality_checks/04_users.py` berfungsi sebagai **integration test khusus untuk memastikan fungsi `clean_users()` bekerja dengan benar ketika diterapkan pada data users yang sebenarnya**. Inputnya adalah `data/raw/users.json` dan fungsi `clean_users()` dari `src.cleaners.py`. Prosesnya membaca data raw, menjalankan `clean_users()`, kemudian memeriksa jumlah data clean dan rejected, conservation antara raw-clean-rejected, keunikan ID, keberadaan U001, normalisasi email U001, perubahan email invalid U003 menjadi NULL, keberadaan alasan rejection, serta keberadaan action DROP, FIX, dan NULL dalam decision log. Outputnya berupa **PASS/FAIL untuk setiap pemeriksaan**, ringkasan jumlah raw/clean/rejected, dan exit code yang menunjukkan apakah integration test berhasil seluruhnya.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python quality_checks/03_helper_contracts.py
PASS text None => None
PASS text trim => Rani
PASS text empty => None
PASS email normalize => rani@mail.com
PASS email invalid => None
PASS price string => 19.5
PASS price negative => None
PASS quantity string => 2
PASS quantity zero => None
PASS quantity decimal => None
PASS quantity bool => None
11/11 helper cases passed
```

8. python quality\_checks/04\_users.py

Script `quality_checks/04_users.py` berfungsi sebagai **integration test khusus untuk memastikan fungsi `clean_users()` bekerja dengan benar ketika diterapkan pada data users yang sebenarnya**. Inputnya adalah `data/raw/users.json` dan fungsi `clean_users()` dari `src.cleaners.py`. Prosesnya membaca data raw, menjalankan `clean_users()`, kemudian memeriksa jumlah data clean dan rejected, conservation antara raw-clean-rejected, keunikan

ID, keberadaan U001, normalisasi email U001, perubahan email invalid U003 menjadi NULL, keberadaan alasan rejection, serta keberadaan action DROP, FIX, dan NULL dalam decision log. Outputnya berupa **PASS/FAIL untuk setiap pemeriksaan**, ringkasan jumlah raw/clean/rejected, dan exit code yang menunjukkan apakah integration test berhasil seluruhnya.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python quality_checks/04_users.py
PASS count clean
PASS count rejected
PASS conservation
PASS unique IDs
PASS U001 ID trimmed
PASS U001 email normalized
PASS U003 invalid email null
PASS reasons present
PASS decision actions

users: raw=16 clean=14 rejected=2
```

9. python quality\_checks/05\_full\_pipeline.py

Script `quality_checks/05_full_pipeline.py` berfungsi sebagai **integration test untuk menguji seluruh fungsi cleaning secara bersama-sama**, yaitu `clean_users()`, `clean_products()`, dan `clean_transactions()`, sehingga memastikan keseluruhan logika cleaning telah memenuhi spesifikasi. Inputnya adalah tiga file raw `users.json`, `products.json`, dan `transactions.json`, beserta target jumlah clean/rejected dan nama field ID yang telah ditentukan dalam SPECS. Prosesnya menjalankan masing-masing cleaner kemudian memeriksa jumlah clean dan rejected, conservation, keberadaan dan keunikan ID, keberadaan alasan rejection, serta kelengkapan struktur decision log. Outputnya berupa **status PASS/FAIL untuk users, products, dan transactions**, dengan exit code 0 jika seluruh integration test berhasil atau 1 jika terdapat bagian yang gagal.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python quality_checks/05_full_pipeline.py
PASS users {'counts': True, 'conservation': True, 'unique IDs': True, 'reasons': True, 'log schema': True}
PASS products {'counts': True, 'conservation': True, 'unique IDs': True, 'reasons': True, 'log schema': True}
PASS transactions {'counts': True, 'conservation': True, 'unique IDs': True, 'reasons': True, 'log schema': True}
```

10. python run\_pipeline.py

Script `run_pipeline.py` berfungsi sebagai **runner/orchestrator yang benar-benar menjalankan proses data cleaning dari raw data hingga menghasilkan data processed**. Inputnya adalah `users.json`, `products.json`, dan `transactions.json` dari folder `data/raw`, serta fungsi cleaner dari `src.cleaners.py`. Prosesnya membaca setiap raw file, menjalankan cleaner yang sesuai, memisahkan hasil menjadi clean dan rejected, menyimpan keduanya ke folder `data/processed`, serta menggabungkan seluruh decision log menjadi satu file `decision_log.json`; script juga menangani cleaner yang belum diimplementasikan dengan menandainya sebagai `PENDING`. Outputnya adalah **file data bersih, file data rejected, dan decision\_log.json di data/processed**, disertai ringkasan jumlah raw/clean/rejected dan status apakah pipeline selesai atau masih incomplete.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python run_pipeline.py
[OK] users: raw=16 clean=14 rejected=2
[OK] products: raw=12 clean=10 rejected=2
[OK] transactions: raw=15 clean=12 rejected=3
```

#### 11. python validate\_delivery.py

Script `validate_delivery.py` berfungsi sebagai **final validation** atau **quality gate** untuk memastikan seluruh hasil pipeline yang sudah disimpan di `data/processed` benar-benar lengkap, konsisten, dan siap direview/dikirim sebagai hasil akhir. Inputnya adalah data raw dan seluruh output processed/rejected, termasuk `users.json`, `products.json`, `transactions.json`, `file rejected`, dan `decision_log.json`. Prosesnya memeriksa keberadaan file output, jumlah clean dan rejected, conservation, keberadaan serta keunikan ID, alasan rejection, validitas hasil tertentu pada users seperti email U001 dan U003, validitas quantity transactions, serta struktur dan action pada decision log (DROP, FIX, NULL). Outputnya berupa **PASS/FAIL** untuk setiap **quality check** dan kesimpulan akhir **READY FOR REVIEW** jika semua pemeriksaan berhasil atau **NOT COMPLETE** beserta bagian yang bermasalah jika masih ada kegagalan.

```
PS D:\Tutorial\Data Engineer\FSB_Data_Quality_Instructor_Pack> python validate_delivery.py
PASS users {'clean count': True, 'rejected count': True, 'conservation': True, 'IDs present': True, 'IDs unique': True, 'reject reasons': True}
PASS products {'clean count': True, 'rejected count': True, 'conservation': True, 'IDs present': True, 'IDs unique': True, 'reject reasons': True}
PASS transactions {'clean count': True, 'rejected count': True, 'conservation': True, 'IDs present': True, 'IDs unique': True, 'reject reasons': True}
PASS user semantics {'U001 exists': True, 'U001 normalized email': True, 'U003 invalid email null': True}
PASS transaction quantities
PASS decision actions/schema {'FIX', 'DROP', 'NULL'}

READY FOR REVIEW
```

Pembelajarannya dari 11 script di atas adalah script 1–2 mempersiapkan dan memahami kondisi awal, script 3–4 menyediakan logika cleaning, script 5–9 menguji logika tersebut, script 10 menjalankan pipeline sebenarnya, dan script 11 memastikan hasil akhirnya memenuhi seluruh persyaratan sebelum dianggap selesai.